

Relatório de Auditoria de Segurança — Bíblia na Arte

Isolamento de tenant, permissões, IDOR, segredos e inputs sem tratamento

DATA

2026-09-08

REPOSITÓRIO

github.com/rilsonjoas/biblia-na-arte

ESCOPO AUDITADO

Monorepo biblia-na-arte (server/ Fastify 5 + Drizzle ORM + Postgres + @fastify/jwt, web/ React + Vite). Escopo: todas as 27 rotas de server/src/routes (8 arquivos), Policies/guards de auth (plugins/jwt-auth.ts, plugins/security.ts), queries do Drizzle (db/queries.ts), fluxo de submissão pública de artistas e upload de imagem, painel admin do app web (contexts, rotas protegidas) e superfície de renderização de conteúdo (Markdown/HTML).

NOTA METODOLÓGICA

Mecanismo de isolamento de tenant/RLS: **não se aplica** — não há conta de usuário final nem dado multi-tenant; é um catálogo curado com um único time de staff (admin/revisor) que compartilha acesso total por design (achado confirmado: todas as rotas de escrita vivem só em admin.ts e submissions.ts, as outras 6 são 100% leitura pública). O equivalente real de fronteira de autorização de dado é o filtro **active = true** (obra removida por auditoria de direitos autorais não pode aparecer em rota pública) — verificado em toda query pública encontrada. "Permissão no navegador" mapeada para o par frontend (user?.role === 'admin', AdminProtectedRoute) × backend (authenticate/requireAdmin do @fastify/jwt, revalidado contra o banco a cada request). IDOR percorrido em toda rota com parâmetro de rota/arquivo. XSS mapeado pra stack real (React web, não app nativo): dangerouslySetInnerHTML, libs de markdown/sanitização e templates HTML no backend.

1 achado — 1 Informativa — **nenhuma vulnerabilidade explorável encontrada**

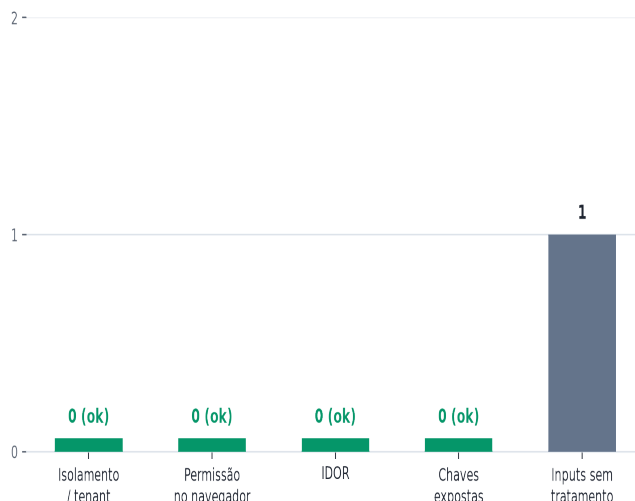
Resumo executivo

Auditoria limpa. Das 5 categorias verificadas em código real, 4 não tiveram nenhum achado (isolamento/RLS não se aplica à stack — sem multi-tenant — e foi verificado o equivalente real; permissão no navegador, IDOR e chaves expostas foram checados rota por rota, sem violação). O único item registrado (B1) é **informativo/housekeeping**: código morto que carrega um padrão sensível mas não é explorável hoje. Nada aqui bloqueia produção nem exige correção urgente.

Achados por severidade



Achados por categoria (verde = nenhum achado, categoria coberta)



Pontos fortes (verificados, com evidência)

✓ **Fronteira pública verificada em toda query** — `eq(artworks.active, true)` (ou `WHERE active` no SQL bruto do Drizzle) aparece em TODA query pública encontrada em `db/queries.ts` — obra removida por auditoria de direitos autorais nunca aparece em rota pública, checado ponto a ponto.

✓ **Parte administrativa 100% isolada das rotas públicas** — As 27 rotas do server vivem em 8 arquivos; escrita (`POST/PATCH/DELETE`) só existe em `admin.ts` (atrás de `authenticate/requireAdmin`) e `submissions.ts` (1 endpoint público, `rate-limited`, só cria linha pendente). As outras 6 são 100% GET público.

✓ **JWT revalidado contra o banco a cada requisição** — `authenticate/requireAdmin` não confia só na assinatura do token — fazem `findUserById` a cada chamada, então apagar um usuário ou trocar seu papel revoga acesso na hora, não só quando o token de 7 dias expirar sozinho.

✓ **Paridade UI (frontend) × guard (backend)** — `canApprove = user?.role === 'admin'` e `AdminProtectedRoute` (`requireAdminRole`) no app web batem 1:1 com `preHandler: app.requireAdmin` nas rotas `/admin/submissions/:id/approve` e `/admin/users` — nenhuma ação `admin-only` depende só da UI escondida.

✓ **Senha com `scrypt` + comparação `constant-time`** — `hashPassword/verifyPassword` (`lib/auth.ts`) usam `scryptSync` com salt aleatório de 16 bytes e `timingSafeEqual` pra comparar o hash — sem vaziar diferença de tempo, sem dependência nova só pra isso.

✓ **Boot falha rápido sem segredo mal configurado** — `config.ts` valida `JWT_SECRET` (mínimo 32 caracteres, sem valor-default) e `DATABASE_URL` via `Zod` no boot — processo sai com `process.exit(1)` e mensagem clara em vez de rodar em produção com config inválida.

✓ **Path traversal bloqueado explicitamente no serving de imagem** — GET /uploads/:filename rejeita qualquer filename com '/' ou '..' antes de montar o path no disco, e só serve do diretório de obras JÁ APROVADAS — imagem de submissão pendente fica numa rota separada, atrás de authenticate.

✓ **XSS coberto por Markdown seguro, com teste próprio provando isso** — Todo texto rico (artwork.description, artwork.classicCommentary, ref.passageText, artist.bio) passa pelo componente Markdown (react-markdown sem plugin rehype-raw, então HTML cru vem escapado por padrão). markdown.test.tsx testa literalmente `<script>alert('x')</script>` e confirma que não executa — verificado no próprio código, não é achado só desta auditoria.

✓ **Consultas parametrizadas mesmo no SQL bruto** — Onde o query builder do Drizzle não bastava (busca full-text, agregações), o SQL usa a tagged template `sql`...${valor}...`` — que parametriza cada interpolação de verdade (via postgres.js). Nenhum uso de `sql.raw()` encontrado no projeto.

✓ **Mensagem de erro de login genérica** — POST /admin/login devolve a mesma mensagem pra e-mail inexistente e senha errada — não dá pra enumerar quais e-mails têm conta só pelo comportamento do endpoint.

✓ **Proteção contra autolockout de administrador** — DELETE /admin/users/:id bloqueia apagar a própria conta logada e bloqueia apagar o último admin restante — sem isso seria possível zerar o painel sem ninguém conseguir recriar acesso.

Pontos fracos

Nenhum risco central de segurança identificado. O item B1 é hardening preventivo de código morto — vale fazer por higiene de repositório, não porque exista exploração hoje.

Achados detalhados

Informativa

B1 · Inputs sem tratamento (equiv. XSS) / housekeeping

web/src/components/ui/chart.tsx:77-95

dangerouslySetInnerHTML em componente shadcn não utilizado + pasta de admin Supabase morta ainda no repositório

ChartStyle usa dangerouslySetInnerHTML pra gerar um <style> a partir de config.theme/config.color (boilerplate padrão do shadcn/ui, instalado mas nunca importado — busca por <Chart em todo web/src/pages e web/src/components não encontrou nenhum uso). Sem exploração possível hoje: não há dado de usuário/API alimentando esse config em lugar nenhum do código atual. Junto: web/src/_archived-supabase-admin/ (auth antiga via Supabase, morta em 2026-08-07 segundo comentário do AdminAuthContext.tsx atual) continua no repositório — confirmado que nenhum arquivo vivo importa dessa pasta (só uma menção em comentário, não import real).

```
// web/src/components/ui/chart.tsx
<style
dangerouslySetInnerHTML={{
  __html: Object.entries(THEMES).map(([theme, prefix]) => `
${prefix} [data-chart=${id}] { ... ${itemConfig.color} ... }
`).join('\n'),
}}
/>
// nunca importado: grep por "<Chart" em src/pages e src/components → 0 resultados
```

Impacto: Nenhum hoje. Se o componente Chart for adotado no futuro com config vindo de dado dinâmico (ex.: nome de tema/cor definido por conteúdo do vault ou input futuro), o dangerouslySetInnerHTML atual injetaria isso sem sanitização numa tag <style> sem revisão adicional — é um código morto com um padrão sensível embutido, não uma vulnerabilidade ativa.

Correção sugerida: Remover o componente Chart não utilizado e a pasta _archived-supabase-admin/ do repositório — reduz superfície de código que precisaria ser reauditado se algum dia for religado por engano. Se decidir manter o Chart pra uso futuro, trocar a geração de CSS por propriedades customizadas via style do React em vez de string HTML crua.

Categorias sem achado (verificadas explicitamente)

✓ **1 — Isolamento de tenant / RLS** — Não se aplica: sem conta de usuário final, sem dado multi-tenant. Equivalente real (active = true) verificado em toda query pública.

✓ **2 — Permissão definida no navegador** — Todo gate de UI (is_owner/role no frontend) tem correspondência exata no backend (authenticate/requireAdmin). Nenhuma ação sensível depende só da UI escondida.

✓ **3 — IDOR** — Todas as 27 rotas revisadas. Escrita só em admin.ts (staff-only, sem conceito de posse por usuário) e submissions.ts (1 endpoint público, cria linha pendente). Path traversal bloqueado em /uploads.

✓ **4 — Chaves expostas** — Nenhum hardcoded. JWT_SECRET sem default (mín. 32 chars, boot falha sem ele), scrypt+timingSafeEqual pra senha, Zod valida env no boot.

Recomendações priorizadas

Prior.	Ação	Por quê
P1	(opcional, baixo esforço) Remover código morto sensível (B1)	chart.tsx (dangerouslySetInnerHTML não usado) e _archived-supabase-admin/ (auth antiga, sem import real) — nenhum dos dois é explorável hoje, mas remover fecha a possibilidade de alguém religar por engano no futuro sem revisão de segurança.

Sem P2/P3 — não há mais nenhuma ação de segurança pendente identificada nesta auditoria.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar. Só 1 issue aqui, de housekeeping — não há achado de segurança acionável que justifique uma issue de correção urgente.

--- ISSUE 1 ---

[Housekeeping] Remover componente shadcn Chart não utilizado e pasta de admin Supabase arquivada

Labels: security, severidade:informativa, housekeeping

****Problema****

Dois itens de código morto encontrados durante auditoria de segurança, nenhum explorável hoje, mas ambos carregam um padrão que merece revisão se algum dia forem religados sem essa auditoria por perto:

****1. `web/src/components/ui/chart.tsx:77-95`**** — ``ChartStyle`` usa ``dangerouslySetInnerHTML`` pra gerar um ``<style>`` a partir de ``config.theme`/`config.color``. Boilerplate padrão do shadcn/ui, instalado mas ****nunca importado**** (``grep "<Chart"`` em todo ``web/src/pages`` e ``web/src/components`` → 0 resultados).

****2. `web/src/\_archived-supabase-admin/`**** — autenticação antiga via Supabase, morta desde 2026-08-07 (comentário em ``AdminAuthContext.tsx`` confirma). Nenhum arquivo vivo importa dessa pasta — só uma menção em comentário.

****Por que vale remover****

Nenhum dos dois é explorável no estado atual (sem dado dinâmico alimentando o Chart, sem rota/import ativo pro admin antigo). Mas código morto com um padrão sensível embutido (``dangerouslySetInnerHTML``, um segundo sistema de auth completo) é o tipo de coisa que passa despercebido numa reintrodução acidental (ex.: alguém copia um exemplo do shadcn docs que usa ``<Chart>`` com config vindo de API, ou reimporta algo do admin antigo achando que é só um componente de UI).

****Evidência****

```
```.tsx
<style
dangerouslySetInnerHTML={{
 __html: Object.entries(THEMES).map(([theme, prefix]) => `...`).join('\n'),
}}
/>
...`
```

**\*\*Impacto\*\***

Nenhum hoje. Risco é só de reintrodução futura sem revisão.

**\*\*Sugestão de correção\*\***

- Remover ``web/src/components/ui/chart.tsx`` (e o import do pacote ``recharts`` associado, se não usado em outro lugar) caso nenhum gráfico esteja planejado no roadmap próximo. Se estiver, trocar a geração de CSS por propriedades customizadas via ``style`` do React.
- Remover ``web/src/_archived-supabase-admin/`` — já cumpriu o papel de referência histórica durante a migração pro JWT próprio.

**\*\*Critérios de aceite\*\***

- [ ] ``chart.tsx`` removido ou sua geração de CSS deixa de usar ``dangerouslySetInnerHTML``.
- [ ] ``_archived-supabase-admin/`` removida do repositório.
- [ ] Build e testes do ``web/`` continuam passando sem os dois.

--- FIM ISSUE 1 ---